



EMI307-1 · ESPECIFICACIÓN DE REQUERIMIENTOS · MÓDULO 03

LAIN — Anexos de la Especificación de Requerimientos

Resumen académico de los Anexos A a F del SRS del sistema LAIN, elaborado conforme al estándar IEEE 830-1998.

Magíster en Ingeniería Informática · UFRO

EMI307-1 · Especificación de Requerimientos · 2026



INSTITUCIÓN ACREDITADA
6 AÑOS
EN TODAS LAS ÁREAS
• GESTIÓN INSTITUCIONAL • DOCENCIA DE PREGRADO
• DOCENCIA DE POSTGRADO • INVESTIGACIÓN
• VINCULACIÓN CON EL MEDIO

¿Qué es el SRS y qué son sus anexos?

LAIN es un sistema de inteligencia de código para asistentes de programación. Construye un mapa de conocimiento del código fuente de un proyecto y lo pone a disposición del Desarrollador y del Asistente de IA.

SRS (*Software Requirements Specification*) es el documento que describe, de manera verificable, qué debe hacer el sistema.

Se elaboró conforme al estándar **IEEE 830-1998**, que define la estructura formal de un SRS. El documento se organiza en tres bloques:

- **Introducción** — propósito, alcance, definiciones, referencias.
- **Descripción general** — perspectiva, funciones, usuarios, restricciones y supuestos.
- **Requisitos específicos** — veintisiete **RF**, diez **RNF** y cinco **restricciones** del sistema, con su matriz de trazabilidad.

Esta especificación integra el trabajo de **actividades previas del módulo** —inscripción del proyecto, identificación de stakeholders, educación de requerimientos, análisis, modelado de casos de uso, modelado del proceso de negocio y diseño preliminar de interfaces—. Cada actividad dejó un antecedente que el SRS recoge y referencia.

El estándar distingue tres categorías de requisito: los **RF** definen el alcance funcional, los **RNF** fijan la calidad esperada y las **restricciones** son condiciones no negociables. En este SRS hay veintisiete RF, diez RNF y cinco restricciones.

LOS SEIS ANEXOS

Qué complementa cada uno

- **A — Diagrama de casos de uso.** Visión gráfica de los CU y sus actores.
- **B — Especificación de CU.** Descripción textual detallada.
- **C — Proceso de negocio.** PN-01, situación actual y propuesta.
- **D — Interfaces.** Wireframes de las pantallas principales.
- **E — Prototipo con IA.** Exploración visual de cuatro vistas.
- **F — Declaración de uso de IA.** Cómo se usó la IA en la entrega.

De la fricción operativa a los RF

El SRS abre con tres elementos: un **problema** observado, unos **stakeholders** identificados (educidos y validados en actividades previas del módulo) y una **necesidad de origen** trazable hasta los requerimientos. Esta slide condensa la Sección 2.3 del documento.

EL PROBLEMA (SRS §1.2)

Fricción con los asistentes de IA

Cuando un asistente razona sobre una base de código grande, solo ve el archivo abierto o resultados de búsqueda textual.

Sin esa visibilidad, sus propuestas rompen código en partes no consideradas, duplican funcionalidad que ya existía y dejan fuera acoplamientos que solo se ven en el historial.

Costo en tokens y baja precisión en las respuestas. El Desarrollador, por su parte, carecía de una herramienta que respondiera con evidencia a preguntas como «si modifico esta función, ¿qué más se ve afectado?».

STAKEHOLDERS (SRS §2.3)

Tres, con influencia desigual

Principal — Sebastián Puentes. Desarrollador y propietario del producto (también cumple el rol de usuario «Desarrollador de software»). Influye en alcance y prioridades.

Secundario — Docente EMI307-1. Evalúa que la especificación sea coherente y trazable. Define los criterios de aceptación del documento.

Terciario — Comunidad. Desarrolladores usuarios de asistentes de IA, cuyas necesidades se recogen indirectamente a través del stakeholder principal.

NECESIDAD DE ORIGEN → RF (SRS §2.3)

Trazabilidad explícita

La necesidad de origen —**reducir el consumo de tokens al entregar contexto al asistente**— aterriza en los **RF-21** y **RF-22** (provisión de contexto curado y acotado) y en el **RF-25** (consulta selectiva del mapa). El **CU-08** registra como observación que el paquete de contexto debe ser acotado, y la **RG-1** exige ejecución local para que el código no salga del equipo del stakeholder.

Por qué importa en el SRS. La Sección 2.3 ata la necesidad de origen de cada stakeholder con los requerimientos que la resuelven. Esa cadena — problema → stakeholder → RF — es el corazón de la trazabilidad.

LAIN en acción: una sesión real

El sistema ya existe y se ejecuta. Esta es una **sesión típica**: inicialización del espacio de trabajo, consulta en lenguaje natural y análisis estructural. La captura muestra los comandos y la salida real del CLI.

```
$ lain init --workspace . --transport stdio --yes

" Detectando asistente...
✓ Asistente detectado: Claude Code
✓ Lenguajes identificados: Rust, TypeScript, Python (3/11)
" Construyendo mapa de conocimiento...
✓ Símbolos indexados: 1.247
✓ Relaciones LSP: 4.832
✓ Mapa persistido en .lain/graph.bin
✓ Integración con Claude Code: ~/.claude/settings.json

$ lain ask "¿Dónde se maneja la autenticación?"

- Buscando símbolos relevantes (semantic + estructural)...
✓ auth/jwt.rs::validate_token    módulo: auth · confianza: alta
✓ auth/session.rs::create_session módulo: auth · confianza: alta
✓ middleware/auth.rs::check_auth módulo: middleware · confianza: alta

$ lain blast-radius --symbol auth/jwt.rs::validate_token --depth 3

- Resolviendo radio de impacto (3 niveles, cierre transitivo)...
✓ 14 símbolos afectados (3 directos, 11 transitivos)
  Nivel 1: middleware/auth.rs::check_auth
            api/users.rs::get_current_user
            api/admin.rs::require_admin
  Nivel 2: api/users.rs::update_user
            tests/auth_test.rs::test_validate
            ... (8 más)

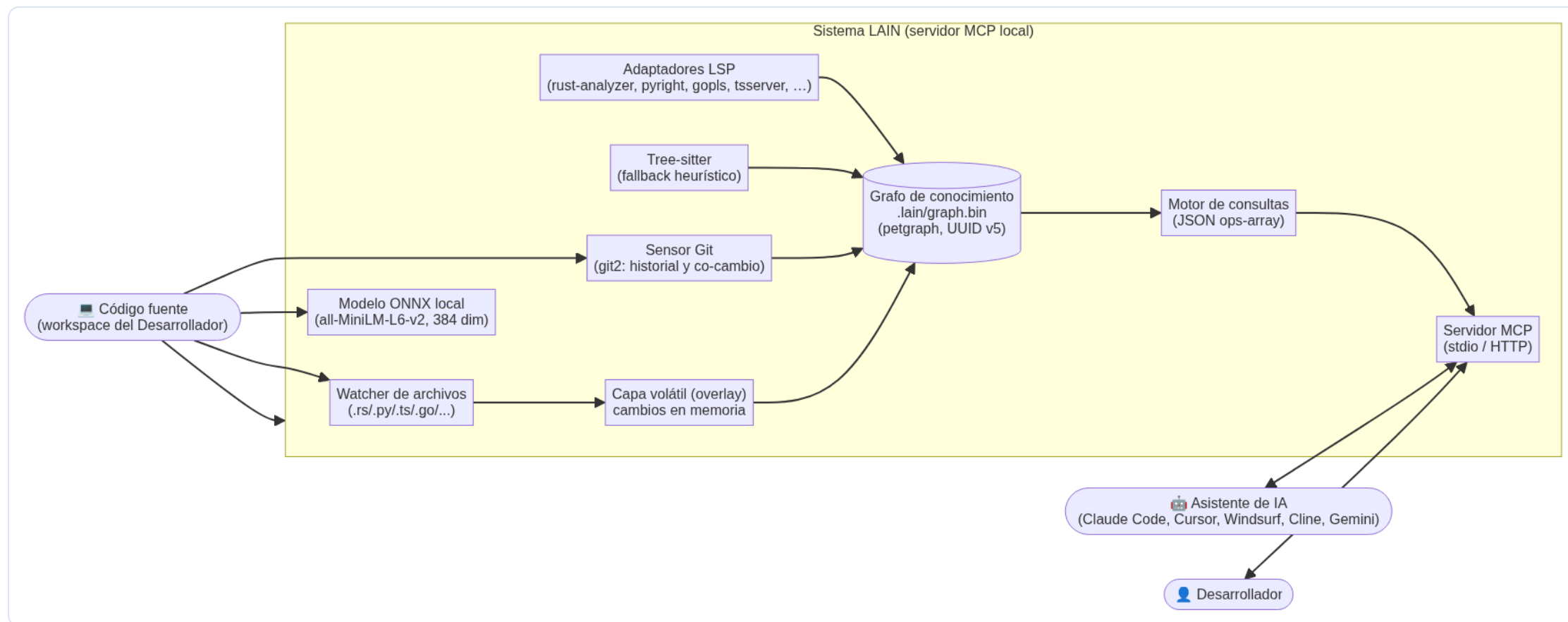
$ lain call-chain --from main --to persist_audit_log

✓ Ruta encontrada (4 pasos):
  main → handle_request → process_event → persist_audit_log
```

Cada comando materializa un caso de uso del Anexo A: `lain init` corresponde a **CU-01**, `lain ask` a **CU-04** (búsqueda semántica), `lain blast radius` a **CU-02** (radio de impacto) y `lain call chain` a **CU-03** (cadena de llamadas).

Cómo está construido LAIN

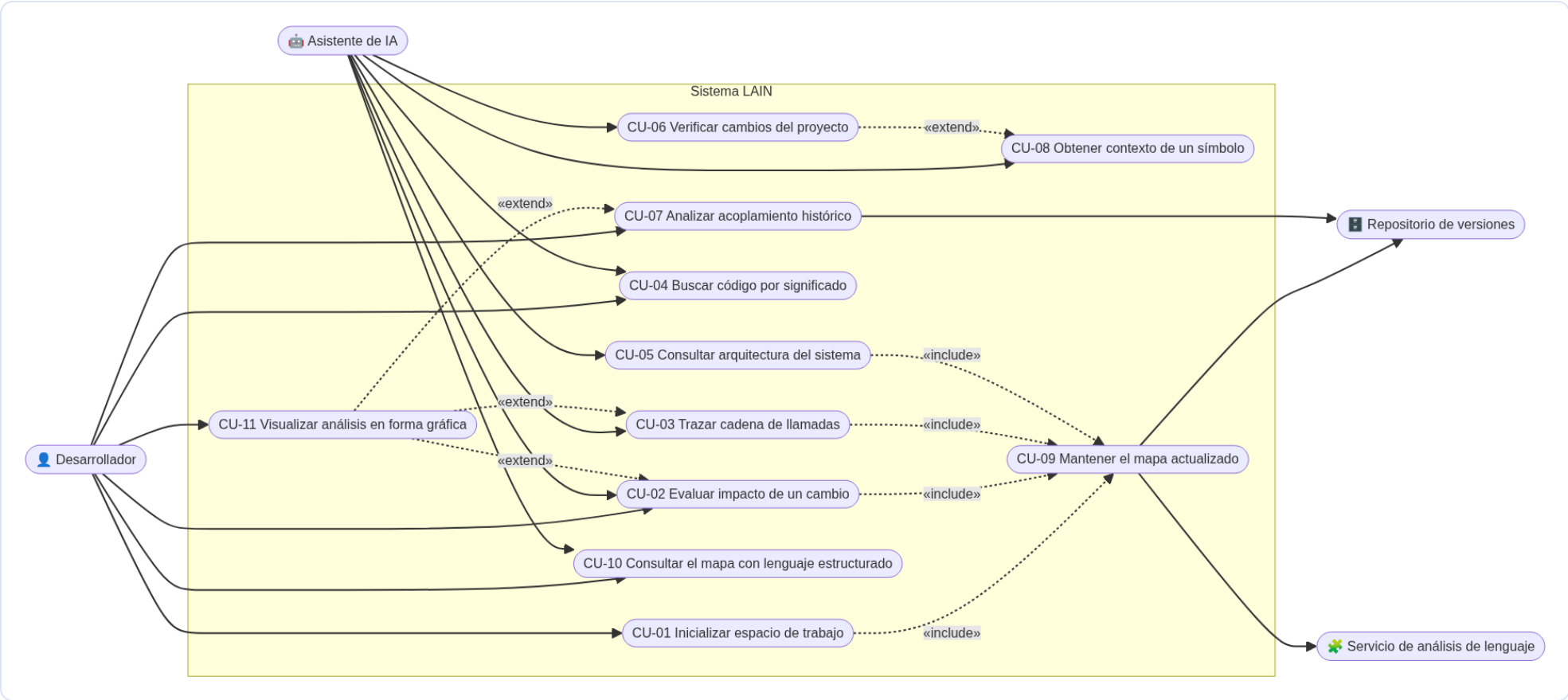
El sistema se ejecuta como un **servidor MCP local** en la máquina del Desarrollador. El grafo de conocimiento se construye a partir del código fuente mediante tres extractores paralelos, y se expone al Asistente de IA por el protocolo MCP estándar.



Trazabilidad. Los extractores (LSP, Tree-sitter, Git) alimentan el grafo de conocimiento (RF-02); el modelo ONNX local sostiene la búsqueda semántica (RF-20); el servidor MCP estandariza la comunicación con los asistentes (RG-2, RG-3).

Casos de uso: actores, límite y alcance

El sistema LAIN se describe mediante **once casos de uso** y **cuatro actores**. El **límite del sistema** está representado por el recuadro «Sistema LAIN»; los actores principales lo cruzan para iniciar CU, y los secundarios son consultados desde dentro.



Relaciones include/extend. CU-09 «Mantener el mapa» es incluido por los casos de consulta (asegura frescura antes de responder). CU-11 «Visualizar» extiende CU-02, CU-03 y CU-07 cuando el Desarrollador solicita una vista gráfica. Coherencia con los veintisiete RF vía matriz de trazabilidad (SRS §3.4).

Especificación de los casos de uso

Los ocho casos principales siguen una **plantilla única**; los tres utilitarios (CU-09, CU-10 y CU-11) se documentan en forma abreviada por su dependencia de los anteriores.

PLANTILLA COMÚN

Estructura aplicada a cada CU

- Identificador y nombre.
- Actor principal y actores secundarios.
- Objetivo del caso de uso.
- Precondiciones.
- Flujo principal (pasos numerados).
- Flujos alternativos o excepciones.
- Postcondiciones.
- RF asociados.
- Observaciones o supuestos pendientes.

EJEMPLO: CU-02 «EVALUAR IMPACTO DE UN CAMBIO»

Resumen del caso

Objetivo: conocer, antes de modificar un símbolo, el conjunto de símbolos afectados directa o transitivamente.

Precondición: espacio de trabajo inicializado y símbolo existente en el mapa.

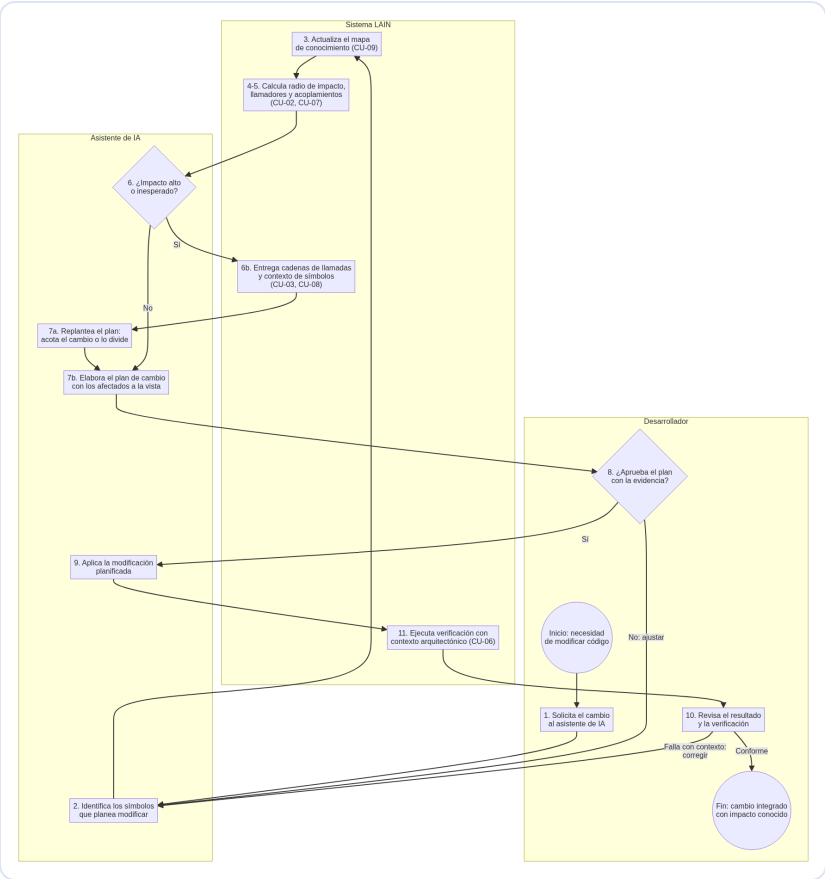
Flujo principal: (1) solicitud → (2) actualización del mapa → (3) resolución de referencias → (4) cierre transitivo → (5) retorno jerarquizado.

RF asociados: RF-07, RF-09, RF-10, RF-11, RF-27.

Detalle en anexos/B-especificacion-casos-de-uso.md.

Proceso de negocio PN-01

PN-01 «Evaluación de impacto antes de modificar código» describe cómo decide y ejecuta un equipo de desarrollo una modificación sobre código existente. El diagrama siguiente corresponde a la **situación propuesta**, con el sistema LAIN incorporado entre los participantes.



Elementos del proceso

- **Inicio:** surge la necesidad de modificar código.
- **Término:** cambio integrado con impacto conocido y verificado.
- **Decisiones:** «¿Impacto alto o inesperado?» y «¿aprueba el plan?».

Principio clave

El sistema acompaña al equipo sin reemplazarlo, aporta evidencias al proceso donde hoy se vive el retrabajo, pero la decisión sigue siendo humana.

Diseño preliminar de interfaces

Se diseñan **cinco wireframes de baja fidelidad** que materializan los casos de uso principales. Las pantallas están dirigidas al Desarrollador; el Asistente de IA interactúa por un protocolo estándar y no requiere GUI.

PANTALLA	NOMBRE	ACTOR	CU	RF
P-01	Asistente de configuración inicial	Desarrollador	CU-01	RF-01, RF-02
P-02	Consola de consultas y estado	Desarrollador	CU-10, CU-09	RF-25, RF-03–06, RF-22
P-03	Visualización de radio de impacto	Desarrollador	CU-02 + CU-11	RF-07, RF-27
P-04	Visualización de cadena de llamadas	Desarrollador	CU-03 + CU-11	RF-08, RF-27
P-05	Mapa de calor de acoplamiento	Desarrollador	CU-07 + CU-11	RF-18, RF-27

Elementos por pantalla. Cada wireframe declara explícitamente: (i) **campos de entrada** (carpeta del proyecto, símbolo consultado, profundidad, ventana temporal); (ii) **acciones disponibles** (ejecutar consulta, sincronizar, enriquecer, exportar); y (iii) **mensajes y validaciones** para los flujos alternativos del Anexo B (símbolo inexistente, servicio de análisis no disponible, historial insuficiente, consulta mal formada, etc.).

Prototipo exploratorio apoyado con IA

Para validar de manera temprana la interacción del Desarrollador, se construyó un **miniprototipo** con cuatro vistas web (P-02 a P-05), generado con apoyo de Claude Code y ajustado por el autor.

FIGURA E-1 · CONSOLA DE CONSULTAS (P-02)



FIGURA E-2 · RADIO DE IMPACTO (P-03)



FIGURA E-3 · CADENA DE LLAMADAS (P-04)



FIGURA E-4 · ACOPLAMIENTO HISTÓRICO (P-05)



Las capturas usan los mismos datos de ejemplo que los wireframes del Anexo D, para facilitar la comparación visual. Detalle en `anexos/E-prototipo-ia.md`.

Limitaciones declaradas. El prototipo opera con datos de ejemplo y cubre solo una parte de los flujos del Anexo B. Quedan fuera de su alcance la pantalla P-01, la interacción del Asistente de IA y la validación de RNF. Las vistas se mantienen en inglés mientras la especificación está en español, y el código generado **no debe reutilizarse** como base de la implementación.

Declaración de uso de Inteligencia Artificial

La IA se usó como **herramienta de apoyo** al proceso de Ingeniería de Requerimientos. Su rol fue acotado: nunca fue fuente definitiva de requerimientos ni sustituto del análisis del autor.

F.1 – F.3

Qué se usó y para qué

- **Herramienta:** Claude Code (Anthropic), única herramienta de IA empleada.
- **Propósito:** apoyar la estructuración del SRS según IEEE 830-1998, la redacción de RF/RNF, la generación de diagramas y la construcción del prototipo.
- **Modalidad:** generación iterativa a partir de descripciones en lenguaje natural.
- **Prompts generales:** entre otros, «elaborar el SRS según IEEE 830-1998», «redactar RF/RNF claros, formales y verificables», «generar el diagrama de CU con relaciones include y extend» y «revisar los pendientes del documento».

F.4 – F.5

Qué generó y qué ajustó el autor

- **Generado:** redacción base, diagramas, wireframes, prototipo.
- **Ajustado por el autor:** autoría individual, stakeholders reales, datos de ejemplo trazables, selección de vistas.
- **Decisiones humanas:** alcance, prioridades, proceso PN-01, decisión de CU-03 (ruta única).

F.7

Aspectos pendientes de validación

- Umbrales cuantitativos de los RNF con stakeholders reales.
- Falsos positivos aceptables en la detección de código sin uso.
- Prueba de usabilidad del proceso de instalación (RNF-06).
- Necesidades del tercer stakeholder (comunidad amplia de desarrolladores).

Anexos cubiertos y referencias

Los seis anexos cubren el sistema desde ángulos distintos: el A desde los casos de uso, el B desde la especificación, el C desde el proceso de negocio, el D y el E desde las interfaces, y el F desde la transparencia sobre el uso de IA.

ANEXO	CONTENIDO	ARCHIVO
A	Diagrama de casos de uso (once CU, cuatro actores)	A-diagrama-casos-de-uso.md
B	Especificación de casos de uso	B-especificacion-casos-de-uso.md
C	Proceso de negocio PN-01 (actual y propuesto)	C-proceso-de-negocio.md
D	Diseño preliminar de interfaces (cinco wireframes)	D-interfaces.md
E	Prototipo apoyado con IA (cuatro vistas)	E-prototipo-ia.md
F	Declaración de uso de IA	F-declaracion-uso-ia.md

Control de versiones. El documento incluye al inicio una tabla con versión, fecha, autor y descripción del cambio, donde la versión final es la 1.0. El historial detallado de la elaboración queda registrado en el repositorio y se consulta con `git log -- docs/srs/`, lo que da cuenta del trabajo realizado a lo largo del módulo conforme lo solicita la pauta.

Integración con actividades previas. El SRS integra el trabajo de las actividades previas del módulo (identificación de stakeholders, educación, análisis, modelado de casos de uso, modelado del proceso de negocio y diseño preliminar de interfaces), y cada sección del documento lo referencia de forma trazable.

Referencias

Anthropic. (s. f.). *Claude Code* [Software de asistencia de programación]. <https://www.anthropic.com/claude-code>

Institute of Electrical and Electronics Engineers. (1998). *IEEE Std 830-1998: IEEE recommended practice for software requirements specifications*. IEEE. <https://doi.org/10.1109/IEEESTD.1998.88286>

Dalpiaz, F., Franch, X., & Horkoff, J. (2016). *iStar 2.0 language guide* (arXiv:1605.07767). arXiv. <https://arxiv.org/abs/1605.07767>

Puentes, S. (2026, julio 7). *Especificación de requerimientos de software: Sistema LAIN — Plataforma de inteligencia de código para asistentes de desarrollo* (Versión 1.0) [Documento SRS]. <https://github.com/spuentesplain/blob/main/docs/srs/SRS.md>